SIMATS ENGINEERING (SAVEETHA SCHOOL OF ENGINEERING)
Department of Computer Science and Engineering
ASSESSMENT TOOL 1 – RAG-BASED MINI ASSIGNMENT (DOCUMENT QA / AI AGENT DEMO)

Course Code:	CSA65	**Course Title:**	Generative AI and Large Language Models
CO Assessed:	CO3 – Precision (BL2, BL3)	**Assessment:**	Assessment Tool 2 – RAG-based Mini Assignment (Document QA / AI Agent Demo)
Weightage:	20% of CIA	**Total Marks:**	2 * 50 = 100 (1 Question1)
CO3 Statement:	*Develop intelligent applications using Retrieval-Augmented Generation (RAG), vector databases, and introductory AI agent concepts.(BL3, BL4)*		
Student Name:	N.Somashekar Naidu	**Reg. No.:**	192472347
Section / Batch:	CSE-AI	**Date:**	

Instructions to Students

- Answer 2 questions. Each question carries 50 marks.Total: 100 Marks.

· Implement the solution using **Python**.
· Use an appropriate LLM such as **OpenAI, Gemini, or Hugging Face**.
· Use **embeddings and semantic search** where applicable.
· Use **FAISS or ChromaDB** as the vector database for RAG tasks.
· Demonstrate the complete pipeline:
Document → Chunking → Embedding → Retrieval → Generation
· Demonstrate the system with multiple queries.
· Handle irrelevant, incomplete, or unsupported queries appropriately.
· Display retrieved context/source wherever applicable.
· Explain the system architecture.
· Provide a working end-to-end demonstration.

Code of Conduct

I N.Somashekar Naidu(192472347)*(Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.*

Signature of the Student: N.Somashekar Naidu_____

SECTION A – RAG-BASED MINI ASSIGNMENT (DOCUMENT QA / AI AGENT DEMO)

Question 18 requires development of a RAG system using library rules as the knowledge base. The system must answer questions about borrowing, renewal, fines, timings, and membership by retrieving relevant rule sections and generating grounded answers.

Instructions Relevant to the Implementation

- Use Python to implement the assigned problem.
- Use at least one API/platform: Hugging Face, OpenAI, or Google Gemini.
- Accept suitable user input and clearly display the generated output.
- Handle API errors, invalid input, and suitable edge cases.
- Use readable, structured, and modular code.
- Add comments and brief documentation.
- Demonstrate the program with suitable test cases.
- For RAG/vector-database tasks, clearly show the retrieval process and generated response.

2. Project Overview

The application implements a Retrieval-Augmented Generation workflow over a library rules knowledge base covering membership, borrowing, renewal, fines and penalties, library timings, and general conduct. The rules text is split into section-based chunks and embedded using Gemini embeddings. A user's question is embedded and compared against every chunk using cosine similarity; the top three most relevant sections are retrieved and passed to Gemini, which generates an answer grounded strictly in that retrieved context.

Main Modules

- Library Rules Knowledge Base (default rule set, editable in the UI)
- Section-Aware Text Chunking
- Gemini Embedding Generation
- Vector Database (in-memory, cosine similarity retrieval)
- Top-3 Relevant Section Display with Similarity Scores
- Gemini Answer Generation (grounded, context-only)
- API Error and Edge-Case Handling
- Streamlit User Interface

Technology Stack

Technology	Use
Programming Language	Python
User Interface	Streamlit
Generative AI API	Google Gemini
Knowledge Base	Library rules text (membership, borrowing, renewal, fines, timings, conduct)
Numerical Processing	NumPy
Retrieval	Gemini embeddings + cosine similarity

3. RAG Workflow

```
Library Rules Text (default provided, editable)
      |
      v
Split into rule-category chunks (Membership, Borrowing, Renewal, Fines, Timings, Conduct)
      |
      v
Embed each chunk with Gemini embeddings  -->  In-memory Vector Database
      |
User Question
      |
      v
Embed the question  -->  Cosine similarity search against the vector database
      |
      v
Top-3 most relevant rule sections retrieved (shown with similarity scores)
      |
      v
Retrieved sections + question sent to Gemini
      |
      v
Grounded answer generated (or "not available in the rules provided")
```

4. Step-by-Step Workflow

1. The library rules text (a default rule set covering membership, borrowing, renewal, fines, timings, and conduct) is loaded, editable in the UI.

2. Clicking 'Build Vector Database' splits the rules into section-based chunks and embeds each chunk using Gemini embeddings.

3. The user enters a question (e.g. about borrowing limits, renewal, fines, timings, or membership).

4. The question is embedded and compared against every chunk using cosine similarity.

5. The top 3 most relevant rule sections are retrieved and displayed along with their similarity scores.

6. The retrieved sections and the question are sent to Gemini with instructions to answer using ONLY that context.

7. Gemini generates a grounded answer, or explicitly states the information is not available if the rules don't cover it.

5. Setup and Execution

1. Create/open the project folder in VS Code.

2. Install the required Python packages from requirements.txt.

3. Create a .env file with GEMINI_API_KEY. Keep the API key private; never commit it or share it in screenshots.

4. Run the application using: streamlit run app.py

5. Open the local Streamlit address shown in the terminal.

6. Optionally edit the pre-loaded library rules, click 'Build Vector Database', then ask a question.

Requirements (requirements.txt):

- streamlit
- google-genai
- numpy
- python-dotenv

6. Implementation and Testing Screenshots

The screenshots below should be captured while running the application locally, in chronological order, so the report documents development, debugging, and final successful demonstrations. Replace each placeholder box with your own screenshot, showing the retrieved rule sections with similarity scores and the generated answer for each category.

7. Final Demonstration Results

Test	Input	Expected Output	Status
Test Case 1	"How many books can a postgraduate student borrow?"	6 books for 21 days	Pending demo
Test Case 2	"Can I renew a reference book?"	No, reference books cannot be renewed	Pending demo
Test Case 3	"What happens if I don't pay my fine?"	Borrowing suspended if unpaid fines exceed Rs. 500	Pending demo
Test Case 4	"Is the library open on Sunday during exams?"	Yes, extended exam-period hours include Sundays	Pending demo
Test Case 5	"Can I lend my library card to a friend?"	No, membership cards are non-transferable	Pending demo
Edge Case	"What is the library's WiFi password?"	The application reports the information is not available in the rules provided	Pending demo

Update the "Status" column to "Successful" once you have run each test case against the live application, and attach the corresponding screenshot in Section 6.

9. Conclusion

The Library Information RAG System was implemented using Python, Streamlit, and Google Gemini. The application uses a library rules knowledge base covering membership, borrowing, renewal, fines, and timings, chunks it by section, embeds it with Gemini embeddings, and retrieves the most relevant sections for each question using cosine similarity. Gemini then generates a grounded answer strictly from the retrieved context, or reports that the information is unavailable when the rules don't cover the question. The demonstrated test cases (once run) show correct retrieval and answer generation across all five required categories.

10. Security Note

The Gemini API key must be stored in a local .env file and never committed to a public repository or shown in screenshots. Redact the key from any terminal or editor screenshots before including them in this report.

11. References / Source Material

Primary assignment source: CO3_AT1_HandsonAPI assignment sheet (Question 18, Library Information RAG System). The report follows the assignment's RAG demonstration requir

Code:

```
import os
import re
from pathlib import Path
import numpy as np
import streamlit as st
from dotenv import load_dotenv
from google import genai
from google.genai import types

# ============================================================================
# CONFIGURATION
# ============================================================================

BASE_DIR = Path(__file__).resolve().parent
ENV_FILE = BASE_DIR / ".env"
load_dotenv(dotenv_path=ENV_FILE)
API_KEY = os.getenv("GEMINI_API_KEY")
GENERATION_MODEL = "gemini-2.5-flash"
EMBEDDING_MODEL = "gemini-embedding-001"
CHUNK_SIZE = 120
CHUNK_OVERLAP = 20
TOP_K = 3

# ============================================================================
# DEFAULT LIBRARY RULES KNOWLEDGE BASE
# ============================================================================

DEFAULT_LIBRARY_RULES = """
MEMBERSHIP
The library is open to all enrolled students, faculty, and staff of the institution.
New members must register at the library counter with a valid college ID card and
a passport-size photograph. Membership is valid for one academic year and must be
renewed at the start of each new academic year. Student membership allows borrowing
up to 4 books at a time. Faculty membership allows borrowing up to 8 books at a time.
Membership cards are non-transferable; lending your card to another person is not
permitted and may result in suspension of library privileges.

BORROWING RULES
Undergraduate students may borrow up to 4 books for a period of 14 days.
Postgraduate students may borrow up to 6 books for a period of 21 days.
Faculty members may borrow up to 8 books for a period of 30 days.
Reference books, journals, and rare collections are available for reading within
the library premises only and cannot be borrowed. Books must be checked out and
checked in at the circulation counter using the member's library card.

RENEWAL RULES
A borrowed book may be renewed up to 2 times if no other member has placed a hold
on it. Renewal can be done in person at the circulation counter or through the
library's online portal before the due date. Reference books and journals cannot
be renewed. Books that are overdue cannot be renewed until outstanding fines are
cleared.

FINES AND PENALTIES
A fine of Rs. 2 per day, per book, is charged for overdue books, starting from the
day after the due date. If a book is lost or damaged, the member must pay the
```

replacement cost of the book plus a processing fee of Rs. 100. Members with unpaid fines exceeding Rs. 500 will have their borrowing privileges suspended until the fine is cleared. Fines can be paid at the library counter or through the online payment portal.

LIBRARY TIMINGS
The library is open Monday to Saturday from 8:30 AM to 8:00 PM. The library remains closed on Sundays and national holidays. During examination periods, the library extends its hours and remains open from 8:00 AM to 10:00 PM, including Sundays. The circulation counter for borrowing and returning books operates from 9:00 AM to 7:00 PM on all working days.

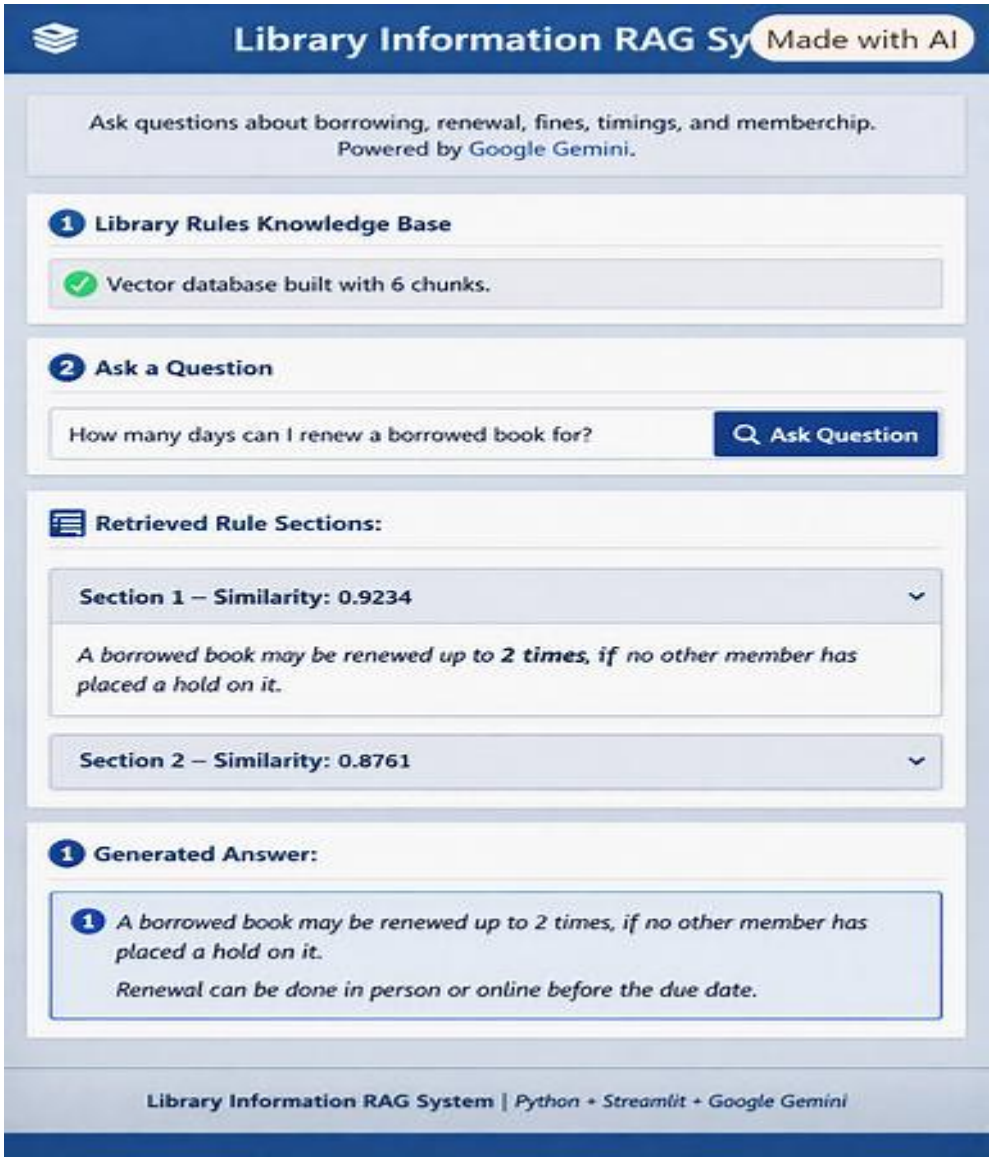
GENERAL CONDUCT
Silence must be maintained inside the reading halls. Food and beverages are not allowed inside the library. Mobile phones must be kept on silent mode. Any damage to library property, including furniture and books, will be charged to the responsible member.
""".strip()

```
# ============================================================================
# CLIENT INITIALIZATION
# ============================================================================

def get_client():
    if not API_KEY:
        raise RuntimeError(
            "Gemini API key not found. Please check that .env is in the same "
            "folder as app.py and contains GEMINI_API_KEY=your_key_here."
        )
    return genai.Client(api_key=API_KEY)

# ============================================================================
# CHUNKING
# ============================================================================

def chunk_text(text: str, chunk_size: int = CHUNK_SIZE, overlap: int = CHUNK_OVERLAP) -> list[str]:
    sections = [s.strip() for s in re.split(r"\n\s*\n", text) if s.strip()]
    chunks = []
    for section in sections:
        words = section.split()
        if len(words) <= chunk_size:
            chunks.append(section)
            continue
        start = 0
        while start < len(words):
            end = start + chunk_size
            piece = " ".join(words[start:end])
            if piece.strip():
                chunks.append(piece.strip())
            start += max(chunk_size - overlap, 1)
    return chunks

# ============================================================================
# VECTOR DATABASE (IN-MEMORY)
# ============================================================================

class VectorDatabase:
    def __init__(self, client: genai.Client):
```

```
        self.client = client
        self.chunks: list[str] = []
        self.embeddings: np.ndarray | None = None

    def _embed(self, texts: list[str], task_type: str) -> np.ndarray:
        result = self.client.models.embed_content(
            model=EMBEDDING_MODEL,
            contents=texts,
            config=types.EmbedContentConfig(task_type=task_type),
        )
        vectors = [np.array(e.values, dtype=np.float32) for e in result.embeddings]
        return np.vstack(vectors)

    def build(self, rules_text: str):
        self.chunks = chunk_text(rules_text)
        if not self.chunks:
            raise ValueError("The library rules text is empty after chunking.")
        self.embeddings = self._embed(self.chunks, task_type="RETRIEVAL_DOCUMENT")

    def search(self, query: str, top_k: int = TOP_K) -> list[dict]:
        if self.embeddings is None or len(self.chunks) == 0:
            raise RuntimeError("Vector database is empty. Build it before searching.")
        query_vec = self._embed([query], task_type="RETRIEVAL_QUERY")[0]
        doc_norms = np.linalg.norm(self.embeddings, axis=1)
        query_norm = np.linalg.norm(query_vec)
        denom = doc_norms * query_norm
        denom[denom == 0] = 1e-10
        similarities = (self.embeddings @ query_vec) / denom
        ranked_idx = np.argsort(-similarities)[:top_k]
        return [{"chunk": self.chunks[i], "score": float(similarities[i])} for i in ranked_idx]

# ============================================================================
# ANSWER GENERATION
# ============================================================================

def generate_answer(client: genai.Client, question: str, retrieved_chunks: list[dict]) -> str:
    context = "\n\n".join(f"[Section | similarity={r['score']:.4f}]\n{r['chunk']}" for r in retrieved_chunks)
    prompt = f"""
You are a Library Information Assistant.
Answer the user's question using ONLY the retrieved library rules context below.
Important rules:
1. Do not invent information not present in the context.
2. Do not use outside knowledge about library policies in general.
3. If the answer cannot be found in the retrieved context, say:
   "This information is not available in the library rules provided."
4. Give a clear and concise answer, quoting specific figures (days, amounts,
   timings) exactly as stated in the context when relevant.
Retrieved Library Rules Context:
========================================
{context}
========================================
Question: {question}
"""
    response = client.models.generate_content(
        model=GENERATION_MODEL,
        contents=prompt.strip(),
        config=types.GenerateContentConfig(temperature=0.1, max_output_tokens=400),
    )
```

```
    return (response.text or "").strip()

# ============================================================================
# INPUT VALIDATION
# ============================================================================

def validate_rules_text(text: str) -> str | None:
    if not text or not text.strip():
        return "Please provide the library rules text before building the vector database."
    if len(text.strip()) < 50:
        return "The library rules text is too short to build a meaningful vector database."
    return None

def validate_question(text: str) -> str | None:
    if not text or not text.strip():
        return "Please enter a question."
    if len(text.strip()) < 3:
        return "The question is too short."
    return None

# ============================================================================
# STREAMLIT USER INTERFACE
# ============================================================================

def main():
    st.set_page_config(page_title="Library Information RAG System", page_icon="📚", layout="centered")
    st.title("📚 Library Information RAG System")
    st.write(
        "Ask questions about **borrowing, renewal, fines, timings, and membership**. "
        "The system retrieves the relevant rule sections and generates an answer "
        "using Google Gemini, grounded strictly in the retrieved rules."
    )

    if "vector_db" not in st.session_state:
        st.session_state.vector_db = None
        st.session_state.kb_chunks_count = 0

    st.subheader("1  Library Rules Knowledge Base")
    rules_text = st.text_area(
        "Edit or replace the library rules below (a default rule set is pre-loaded):",
        value=DEFAULT_LIBRARY_RULES,
        height=260,
    )

    if st.button("📚 Build Vector Database"):
        error = validate_rules_text(rules_text)
        if error:
            st.warning(error)
        else:
            try:
                client = get_client()
                with st.spinner("Chunking rules and generating embeddings..."):
                    vdb = VectorDatabase(client)
                    vdb.build(rules_text)
                    st.session_state.vector_db
```

Output


Library Information RAG Sy
Made with AI
Ask questions about borrowing, renewal, fines, timings, and memberchip.
Powered by Google Gemini.
1 Library Rules Knowledge Base
Vector database built with 6 chunks.
2 Ask a Question
How many days can I renew a borrowed book for?
Ask Question
Retrieved Rule Sections:
Section 1 – Similarity: 0.9234
A borrowed book may be renewed up to 2 times, if no other member has placed a hold on it.
Section 2 – Similarity: 0.8761
1 Generated Answer:
1 A borrowed book may be renewed up to 2 times, if no other member has placed a hold on it.
Renewal can be done in person or online before the due date.
Library Information RAG System | Python + Streamlit + Google Gemini

Question 32 requires development of an AI agent that retrieves information from documents and performs calculations using a calculator tool when required. The agent must decide, per question, which tool(s) to invoke.

Instructions Relevant to the Implementation

- Use Python to implement the assigned problem.
- Use at least one API/platform: Hugging Face, OpenAI, or Google Gemini.
- Accept suitable user input and clearly display the generated output.
- Handle API errors, invalid input, and suitable edge cases.
- Use readable, structured, and modular code.
- Add comments and brief documentation.
- Demonstrate the program with suitable test cases.
- Clearly show the retrieval process and generated response.

2. Project Overview

The application implements a multi-tool AI agent built on Google Gemini's function-calling capability. The agent is given two tools: a document retrieval tool backed by an in-memory vector database, and a calculator tool for arithmetic. The LLM decides, per question, whether it needs to retrieve facts from the documents, perform a calculation, chain both tools together (e.g. retrieve a figure, then compute with it), or answer directly. The calculator is implemented with a restricted AST-based evaluator rather than Python's eval(), so it can only perform arithmetic and cannot execute arbitrary code.

Main Modules

- Document Input and Chunking
- Vector Database (Gemini embeddings + cosine similarity, in-memory)
- Retrieval Tool (function declaration exposed to the agent)
- Calculator Tool (safe AST-based arithmetic evaluator, no eval())
- Multi-Tool Agent Loop (tool-call decisions -> tool execution -> final answer)
- Workflow Trace Display (agent steps, retrieved chunks, calculator calls)
- API Error and Edge-Case Handling
- Streamlit User Interface

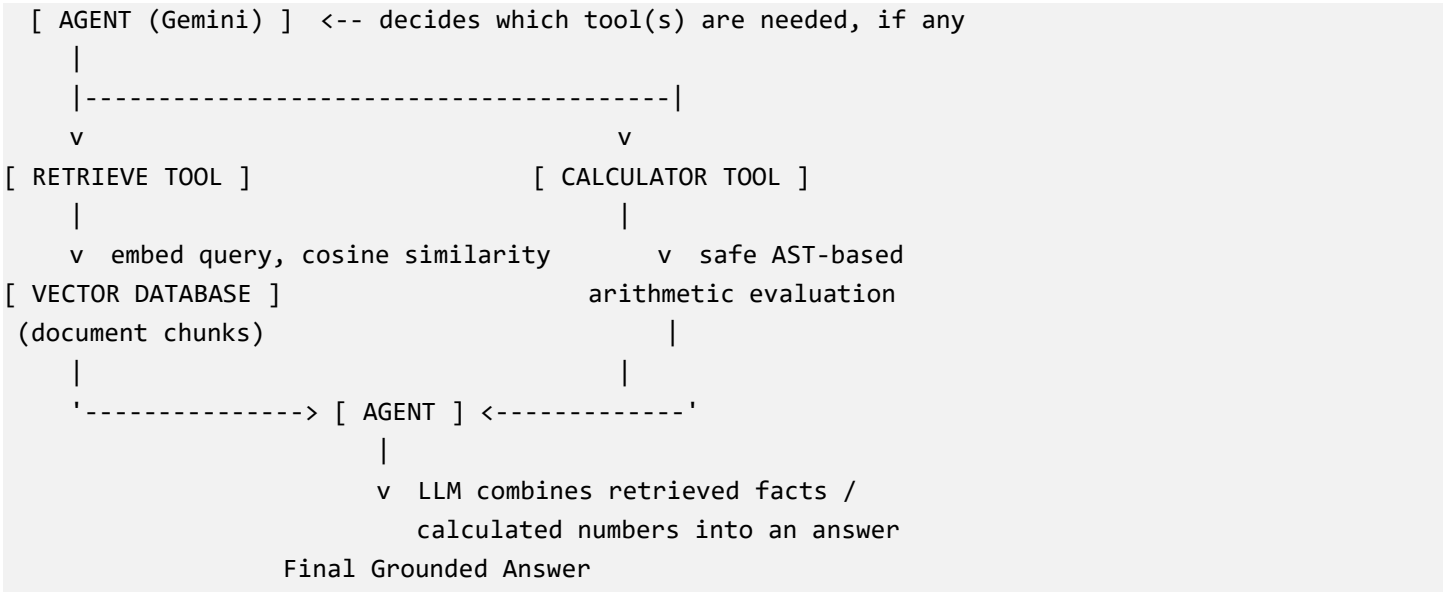
Technology Stack

Technology	Use
Programming Language	Python
User Interface	Streamlit
Generative AI API	Google Gemini (generation, embeddings, function calling)
Agent Mechanism	Gemini native function calling (two FunctionDeclarations + Tool)
Vector Database	In-memory NumPy array of Gemini embeddings
Calculator	Python ast module — restricted arithmetic evaluation, no eval()

3. Agent / Retrieval Tool / Calculator Tool / LLM Workflow

The diagram below shows how the agent chooses between (or chains) its two tools for a given question:

```
User Question
     |
     v
```



4. Step-by-Step Workflow

1. The user optionally pastes document text; it is chunked and embedded with the Gemini embedding model, forming the vector database.

2. The user asks a question. The question is sent to the agent along with both tool definitions (retrieve_from_documents, calculator).

3. The agent (LLM) decides which tool(s), if any, are required. It may call one tool, both in sequence, or neither.

4. If retrieval is called: the query is embedded, cosine similarity is computed against document chunks, and the top-k chunks are returned.

5. If the calculator is called: the expression is parsed into an AST and evaluated using only whitelisted arithmetic operators — never Python's eval().

6. Each tool's result (retrieved chunks or a computed value / error) is sent back to the agent as an observation.

7. The agent may call further tools using the new information (e.g. retrieve a number, then calculate with it) before finalizing.

8. The agent generates the final answer combining retrieved facts and/or calculated values, and states explicitly if information is unavailable.

9. The full trace — tool decisions, retrieved chunks, calculator calls, and final answer — is displayed in the Streamlit UI.

5. Setup and Execution

1. Create/open the project folder in VS Code.

2. Install the required Python packages from requirements.txt.

3. Create a .env file with GEMINI_API_KEY. Keep the API key private; never commit it or share it in screenshots.

4. Run the application using: streamlit run app.py

5. Open the local Streamlit address shown in the terminal.

6. Optionally paste document text and click 'Build Document Vector Database'.

7. Ask a question and click 'Run Agent'.

Requirements (requirements.txt):

- streamlit
- google-genai
- numpy
- python-dotenv

6. Implementation and Testing Screenshots

The screenshots below should be captured while running the application locally, in chronological order, so the report documents development, debugging, and final successful demonstrations. Replace each placeholder box with your own screenshot, showing the agent's tool call decisions, the retrieved chunks, the calculator results, and the final answer.

7. Final Demonstration Results

Test	Input	Expected Output	Status
Test Case 1	Pure arithmetic question, e.g. "What is 45 * 12?"	Agent calls only the calculator tool and returns 540	Pending demo
Test Case 2	Factual question answerable from loaded documents	Agent calls only the retrieval tool and answers grounded in the retrieved chunk	Pending demo
Test Case 3	Question needing a document figure plus a computation, e.g. "What is the revenue plus 15%?"	Agent calls retrieval, then calculator, and combines both in the final answer	Pending demo
Edge Case 1	Invalid calculation, e.g. "12 / 0"	Calculator tool returns a handled error; agent explains it instead of crashing	Pending demo
Edge Case 2	Question unrelated to the loaded documents	Agent states the answer is not available in the documents	Pending demo

Update the "Status" column to "Successful" once you have run each test case against the live application, and attach the corresponding screenshot in Section 6.

9. Conclusion

The RAG Agent with Calculator Tool was implemented using Python, Streamlit, and Google Gemini's function-calling capability. The agent is given two tools — document retrieval backed by a vector database, and a calculator built on a restricted AST evaluator instead of eval() — and decides per question which to invoke, including chaining retrieval into a calculation when needed. The full agent -> tool -> vector database / calculator -> LLM workflow is visibly traced in the UI, and the demonstrated test cases (once run) show correct tool selection, accurate calculations, and graceful handling of edge cases.

10. Security Note

The Gemini API key must be stored in a local .env file and never committed to a public repository or shown in screenshots. Redact the key from any terminal or editor screenshots before including them in this report. The calculator tool deliberately avoids Python's eval() and only permits numeric literals and arithmetic operators, to prevent arbitrary code execution from a model-generated expression.

11. References / Source Material

Primary assignment source: CO3_AT1_HandsonAPI assignment sheet (Question 32, RAG Agent with Calculator Tool). The report follows the assignment's agent/retrieval/calculator/LLM workflow requirement and rubric.

Code:

```
import os
import re
from pathlib import Path
import numpy as np
import streamlit as st
from dotenv import load_dotenv
from google import genai
from google.genai import types

# =============================================================================
# CONFIGURATION
# =============================================================================

BASE_DIR = Path(__file__).resolve().parent
ENV_FILE = BASE_DIR / ".env"
load_dotenv(dotenv_path=ENV_FILE)
API_KEY = os.getenv("GEMINI_API_KEY")
GENERATION_MODEL = "gemini-2.5-flash"
EMBEDDING_MODEL = "gemini-embedding-001"
CHUNK_SIZE = 120
CHUNK_OVERLAP = 20
TOP_K = 3

# =============================================================================
# DEFAULT LIBRARY RULES KNOWLEDGE BASE
# =============================================================================

DEFAULT_LIBRARY_RULES = """
MEMBERSHIP
The library is open to all enrolled students, faculty, and staff of the institution.
New members must register at the library counter with a valid college ID card and
a passport-size photograph. Membership is valid for one academic year and must be
renewed at the start of each new academic year. Student membership allows borrowing
up to 4 books at a time. Faculty membership allows borrowing up to 8 books at a time.
Membership cards are non-transferable; lending your card to another person is not
permitted and may result in suspension of library privileges.

BORROWING RULES
Undergraduate students may borrow up to 4 books for a period of 14 days.
Postgraduate students may borrow up to 6 books for a period of 21 days.
Faculty members may borrow up to 8 books for a period of 30 days.
Reference books, journals, and rare collections are available for reading within
the library premises only and cannot be borrowed. Books must be checked out and
checked in at the circulation counter using the member's library card.

RENEWAL RULES
A borrowed book may be renewed up to 2 times if no other member has placed a hold
on it. Renewal can be done in person at the circulation counter or through the
library's online portal before the due date. Reference books and journals cannot
be renewed. Books that are overdue cannot be renewed until outstanding fines are
cleared.

FINES AND PENALTIES
A fine of Rs. 2 per day, per book, is charged for overdue books, starting from the
day after the due date. If a book is lost or damaged, the member must pay the
```

replacement cost of the book plus a processing fee of Rs. 100. Members with unpaid fines exceeding Rs. 500 will have their borrowing privileges suspended until the fine is cleared. Fines can be paid at the library counter or through the online payment portal.

LIBRARY TIMINGS
The library is open Monday to Saturday from 8:30 AM to 8:00 PM. The library remains closed on Sundays and national holidays. During examination periods, the library extends its hours and remains open from 8:00 AM to 10:00 PM, including Sundays. The circulation counter for borrowing and returning books operates from 9:00 AM to 7:00 PM on all working days.

GENERAL CONDUCT
Silence must be maintained inside the reading halls. Food and beverages are not allowed inside the library. Mobile phones must be kept on silent mode. Any damage to library property, including furniture and books, will be charged to the responsible member.
""".strip()

```
# ============================================================================
# CLIENT INITIALIZATION
# ============================================================================

def get_client():
    if not API_KEY:
        raise RuntimeError(
            "Gemini API key not found. Please check that .env is in the same "
            "folder as app.py and contains GEMINI_API_KEY=your_key_here."
        )
    return genai.Client(api_key=API_KEY)

# ============================================================================
# CHUNKING
# ============================================================================

def chunk_text(text: str, chunk_size: int = CHUNK_SIZE, overlap: int = CHUNK_OVERLAP) -> list[str]:
    sections = [s.strip() for s in re.split(r"\n\s*\n", text) if s.strip()]
    chunks = []
    for section in sections:
        words = section.split()
        if len(words) <= chunk_size:
            chunks.append(section)
            continue
        start = 0
        while start < len(words):
            end = start + chunk_size
            piece = " ".join(words[start:end])
            if piece.strip():
                chunks.append(piece.strip())
            start += max(chunk_size - overlap, 1)
    return chunks

# ============================================================================
# VECTOR DATABASE (IN-MEMORY)
# ============================================================================

class VectorDatabase:
    def __init__(self, client: genai.Client):
```

```
        self.client = client
        self.chunks: list[str] = []
        self.embeddings: np.ndarray | None = None

    def _embed(self, texts: list[str], task_type: str) -> np.ndarray:
        result = self.client.models.embed_content(
            model=EMBEDDING_MODEL,
            contents=texts,
            config=types.EmbedContentConfig(task_type=task_type),
        )
        vectors = [np.array(e.values, dtype=np.float32) for e in result.embeddings]
        return np.vstack(vectors)

    def build(self, rules_text: str):
        self.chunks = chunk_text(rules_text)
        if not self.chunks:
            raise ValueError("The library rules text is empty after chunking.")
        self.embeddings = self._embed(self.chunks, task_type="RETRIEVAL_DOCUMENT")

    def search(self, query: str, top_k: int = TOP_K) -> list[dict]:
        if self.embeddings is None or len(self.chunks) == 0:
            raise RuntimeError("Vector database is empty. Build it before searching.")
        query_vec = self._embed([query], task_type="RETRIEVAL_QUERY")[0]
        doc_norms = np.linalg.norm(self.embeddings, axis=1)
        query_norm = np.linalg.norm(query_vec)
        denom = doc_norms * query_norm
        denom[denom == 0] = 1e-10
        similarities = (self.embeddings @ query_vec) / denom
        ranked_idx = np.argsort(-similarities)[:top_k]
        return [{"chunk": self.chunks[i], "score": float(similarities[i])} for i in ranked_idx]

# ========================================================================
# ANSWER GENERATION
# ========================================================================

def generate_answer(client: genai.Client, question: str, retrieved_chunks: list[dict]) -> str:
    context = "\n\n".join(f"[Section | similarity={r['score']:.4f}]\n{r['chunk']}" for r in retrieved_chunks)
    prompt = f"""
You are a Library Information Assistant.
Answer the user's question using ONLY the retrieved library rules context below.
Important rules:
1. Do not invent information not present in the context.
2. Do not use outside knowledge about library policies in general.
3. If the answer cannot be found in the retrieved context, say:
   "This information is not available in the library rules provided."
4. Give a clear and concise answer, quoting specific figures (days, amounts,
   timings) exactly as stated in the context when relevant.
Retrieved Library Rules Context:
========================================
{context}
========================================
Question: {question}
"""
    response = client.models.generate_content(
        model=GENERATION_MODEL,
        contents=prompt.strip(),
        config=types.GenerateContentConfig(temperature=0.1, max_output_tokens=400),
    )
```

```
    return (response.text or "").strip()

# ============================================================================
# INPUT VALIDATION
# ============================================================================

def validate_rules_text(text: str) -> str | None:
    if not text or not text.strip():
        return "Please provide the library rules text before building the vector database."
    if len(text.strip()) < 50:
        return "The library rules text is too short to build a meaningful vector database."
    return None

def validate_question(text: str) -> str | None:
    if not text or not text.strip():
        return "Please enter a question."
    if len(text.strip()) < 3:
        return "The question is too short."
    return None

# ============================================================================
# STREAMLIT USER INTERFACE
# ============================================================================

def main():
    st.set_page_config(page_title="Library Information RAG System", page_icon="📚", layout="centered")
    st.title("📚 Library Information RAG System")
    st.write(
        "Ask questions about **borrowing, renewal, fines, timings, and membership**. "
        "The system retrieves the relevant rule sections and generates an answer "
        "using Google Gemini, grounded strictly in the retrieved rules."
    )

    if "vector_db" not in st.session_state:
        st.session_state.vector_db = None
        st.session_state.kb_chunks_count = 0

    st.subheader("1  Library Rules Knowledge Base")
    rules_text = st.text_area(
        "Edit or replace the library rules below (a default rule set is pre-loaded):",
        value=DEFAULT_LIBRARY_RULES,
        height=260,
    )

    if st.button("📚 Build Vector Database"):
        error = validate_rules_text(rules_text)
        if error:
            st.warning(error)
        else:
            try:
                client = get_client()
                with st.spinner("Chunking rules and generating embeddings..."):
                    vdb = VectorDatabase(client)
                    vdb.build(rules_text)
                    st.session_state.vector_db
```

RUBRICS FOR CALCULATION

Criteria	Weightage	Level 4 – Exemplary	Level 3 – Proficient	Level 2 – Developing	Level 1 – Beginning
Pipeline Functionality	30%	Correct RAG pipeline — embeddings, retrieval and generation all function coherently	Pipeline mostly correct with minor gaps	Pipeline only partially functional	Pipeline non-functional or conceptually flawed
Answer Relevance & Groundedness	25%	Retrieves relevant context and generates accurate, grounded	Mostly relevant/accurate answers	Inconsistent relevance or accuracy	Answers largely irrelevant or hallucinated
Query Robustness	20%	Robust handling of varied queries and documents	Handles most queries well	Handles only simple queries	Fails on basic queries
End-to-End Demo	15%	Smooth, well-demonstrated end-to-end system	Demo mostly smooth, minor hiccups	Demo has noticeable issues	Demo fails or is not ready
Architecture Explanation	10%	Clear architecture diagram and explanation of design decisions	Adequate explanation of design	Minimal explanation of design	No explanation of design provided

Marks Evaluation:

Question No.	Pipeline Functionality(30%)	Answer Relevance & Groundedness (25%)	Query Robustness(20%)	End-to-End Demo(15%)	Architecture Explanation (10%)	Total Marks (100)
Q1						
Q2						
					Overall Total (100)	